

AAPL

RETURN FORECASTING

Decision trees, ensemble regression
and historical financial data



RESEARCH REPORT

Coursework portfolio edition | 2026

METHODS

Linear regression / Decision tree / Random forest
Gradient boosting / Chronological evaluation

Research overview

RESEARCH QUESTION

Can tree-based regression and ensemble methods learn useful short-horizon signals from historical daily stock-market data?

DATA AND FEATURE ENGINEERING

The original coursework studies Apple (AAPL) daily quotations from 2015-2024. Features include rolling-price ratios, daily returns, price range, lagged returns, and trading volume. The train/test split preserves chronological order.

EXPERIMENT

The report compares Linear Regression, Decision Tree, Random Forest, and Gradient Boosting regressors implemented with Python, pandas, NumPy, scikit-learn, and Matplotlib. Evaluation uses MAE and RMSE.

RESULT REPORTED IN THE COURSEWORK

Random Forest had the lowest reported MAE (1.010%) and RMSE (1.356%). The report also documents limited predictive power and forecasts concentrated near zero. These are historical reported results, not independently reproduced values for this public edition.

PUBLICATION AND REPRODUCIBILITY NOTE

The original academic cover and signed assignment pages were removed, and this English cover and guide were added. The Russian research text and its original page numbers (3-27) are retained. The companion GitHub repository contains a separately maintained reproducible implementation; the report and repository are not asserted to be byte-for-byte identical experiments.

Оглавление

Введение	4
1. Теоретические основы алгоритмов решающих деревьев.....	5
1.1. Обзор методов прогнозирования стоимости активов	5
1.2. Решающие деревья в задачах машинного обучения	6
1.3. Ручной расчёт дерева решений методом ID3	8
1.4. Библиотеки Python для реализации алгоритмов машинного обучения	11
2. Практическая реализация алгоритмов прогнозирования стоимости активов	13
2.1. Подготовка данных и описание датасета.....	13
2.2. Реализация и оценка моделей на Python	15
2.3. Нормализация данных и её влияние на качество моделей	17
2.4. Сравнительный анализ моделей.....	18
Заключение.....	22
Список использованных источников	23
Приложение 1. Листинг программы	25

Введение

В последние годы финансовые рынки становятся всё более непредсказуемыми. Цены активов меняются под влиянием десятков одновременно действующих факторов, и классический технический анализ всё хуже справляется с задачей прогнозирования. На этом фоне методы машинного обучения выглядят значительно перспективнее — они способны находить скрытые закономерности в исторических данных и строить прогнозы, недоступные традиционным подходам.

В данной курсовой работе рассматриваются алгоритмы на основе решающих деревьев в контексте оценки и прогнозирования стоимости финансовых активов. В отличие от нейросетевых моделей, решающие деревья позволяют визуализировать логику прогноза. Такая прозрачность критически необходима в финансовых задачах, где требуется обосновать, почему модель выдала именно тот или иной результат.

Цель курсовой работы — разработать и исследовать алгоритмы прогнозирования стоимости активов на основе решающих деревьев.

Для достижения поставленной цели были решены следующие задачи: изучить математические основы решающих деревьев и алгоритма ID3; провести ручной расчёт дерева решений на модельных данных; подобрать и подготовить датасет финансовых активов; реализовать модели `DecisionTreeRegressor` и `RandomForestRegressor` на языке Python; оценить точность моделей с помощью метрик MAE, RMSE; визуализировать результаты прогнозирования.

Объект курсовой - финансовые активы фондового рынка. Предмет - алгоритмы машинного обучения на основе решающих деревьев.

Структура работы включает введение, два тематических раздела, заключение, перечень использованных источников и приложение с исходным кодом реализованной программы.

1. Теоретические основы алгоритмов решающих деревьев

1.1. Обзор методов прогнозирования стоимости активов

Задача прогнозирования стоимости финансовых активов относится к классу задач регрессии на временных рядах и решается различными методами — от классических статистических подходов до современных алгоритмов машинного обучения.

Наиболее простым и широко применяемым классическим методом является линейная регрессия. Она предполагает, что целевая переменная линейно зависит от набора признаков, что позволяет легко интерпретировать результаты. Однако финансовые временные ряды обладают нелинейной динамикой, наличием выбросов и структурными сдвигами, что существенно ограничивает применимость линейных моделей на практике.

Модели авторегрессии (AR, ARIMA) учитывают временную зависимость значений ряда, однако также строятся на предположении о линейности и стационарности, которое для большинства финансовых инструментов не выполняется на длительных горизонтах.

Нейронные сети, в особенности рекуррентные архитектуры LSTM, способны улавливать сложные нелинейные зависимости в последовательных данных и показывают высокое качество на задачах прогнозирования временных рядов [6]. Вместе с тем нейронные сети требуют значительных вычислительных ресурсов, большого объёма обучающих данных и практически не поддаются интерпретации — невозможно объяснить, почему модель выдала именно такой прогноз.

Алгоритмы на основе решающих деревьев занимают промежуточное положение: они способны моделировать нелинейные зависимости, устойчивы к выбросам и пропускам признаков, не требуют нормализации данных и при этом остаются интерпретируемыми. Ансамблевый метод Random Forest дополнительно снижает дисперсию ошибки за счёт усреднения множества деревьев.

Таким образом, для решения поставленной задачи в данной курсовой работе выбраны алгоритмы на основе решающих деревьев как наиболее подходящие по сочетанию интерпретируемости, устойчивости и способности моделировать нелинейные зависимости.

1.2. Решающие деревья в задачах машинного обучения

Решающее дерево — это один из наиболее наглядных алгоритмов машинного обучения с учителем. Структура алгоритма напоминает перевернутое дерево: в корне находится самый важный признак, от него отходят ветви по значениям этого признака, и так далее вглубь, до конечных узлов (листьев), в которых хранится итоговый ответ модели. Такая структура делает алгоритм легко интерпретируемым: можно буквально проследить путь от входных данных до прогноза.

В задачах прогнозирования стоимости активов решающие деревья применяются как регрессионные модели — то есть на выходе они выдают не класс, а конкретное числовое значение (например, цену акции на следующий день). Для построения регрессионного дерева используется алгоритм CART (Classification and Regression Trees), который на каждом шаге делит обучающую выборку на два подмножества так, чтобы минимизировать среднеквадратичную ошибку внутри каждого из них:

$$\text{MSE} = \left(\frac{1}{n}\right) \cdot \sum_{i=1}^n (y_i - \bar{y})^2.$$

где y_i - истинное значение целевой переменной, $\bar{y}$ - среднее значение по узлу, n - количество наблюдений в узле.

Для задач классификации (например, предсказание направления движения цены: вырастет или упадёт) применяется алгоритм ID3, в основе которого лежит понятие энтропии. Энтропия измеряет степень неопределённости в наборе данных и вычисляется по формуле:

$$E(S) = - \sum_{i=1}^n p_i \cdot \log_2(p_i).$$

где p_i - доля объектов класса i в множестве S , n - количество классов.

При нулевой энтропии все объекты узла принадлежат одному классу - узел чистый, дальнейшее деление не требуется. При максимальной энтропии классы распределены поровну - узел самый «грязный» и требует разбиения.

Для выбора оптимального признака разбиения вычисляется прирост информации (Information Gain):

$$IG(S, A) = E(S) - \sum_{v \in V} \left(\frac{|S_v|}{|S|} \right) \cdot E(S_v).$$

где A - рассматриваемый признак, V - множество его значений, S_v - подмножество объектов со значением признака v . Признак с наибольшим IG выбирается в качестве узла дерева.

Одиночное решающее дерево склонно к переобучению: на обучающей выборке оно работает отлично, а на новых данных - значительно хуже. Для решения этой проблемы применяется ансамблевый метод Random Forest (случайный лес). Идея проста: строится не одно дерево, а несколько сотен независимых деревьев, каждое из которых обучается на случайной подвыборке данных и использует случайное подмножество признаков. Итоговый прогноз формируется как среднее по всем деревьям. Это существенно снижает дисперсию ошибки и повышает устойчивость модели [3].

Для оценки качества регрессионных моделей используются три основные метрики.

Средняя абсолютная ошибка (MAE) показывает среднее отклонение прогноза от реального значения в тех же единицах, что и целевая переменная:

$$MAE = \left(\frac{1}{n} \right) \cdot \sum_{i=1}^n |y_i - \hat{y}_i|$$

Среднеквадратичная ошибка (RMSE) сильнее штрафует за большие отклонения, что делает её чувствительной к выбросам:

$$RMSE = \sqrt{\left[\left(\frac{1}{n}\right) \cdot \sum_{i=1}^n (y_i - \hat{y}_i)^2\right]}.$$

Все две метрики в совокупности дают полную картину качества модели: MAE отражает типичную ошибку, RMSE - чувствительность к выбросам.

1.3. Ручной расчёт дерева решений методом ID3

Для демонстрации работы алгоритма ID3 рассмотрим модельную задачу: по историческим данным о торговом дне требуется предсказать, вырастет ли цена актива на следующий день. Исходные данные представлены в таблице 1.

Таблица 1 - Исходные данные

№	Объем торгов	Тренд	Волатильность	Рост цены
1	Высокий	Бычий	Низкая	Да
2	Высокий	Бычий	Высокая	Да
3	Низкий	Бычий	Низкая	Да
4	Высокий	Медвежий	Высокая	Нет
5	Низкий	Медвежий	Высокая	Нет
6	Низкий	Медвежий	Низкая	Нет
7	Высокий	Бычий	Высокая	Да
8	Низкий	Медвежий	Высокая	Нет
9	Высокий	Медвежий	Низкая	Да
10	Низкий	Бычий	Высокая	Да

Из 10 наблюдений 6 относятся к классу «Да» и 4 - к классу «Нет».

Шаг 1. Вычисление энтропии корневого узла

$$E(S) = -\left(\frac{6}{10}\right) \cdot \log_2\left(\frac{6}{10}\right) - \left(\frac{4}{10}\right) \cdot \log_2\left(\frac{4}{10}\right) \approx 0,971.$$

Шаг 2. Прирост информации для признака «Тренд»

При Тренд = «Бычий»: 5 наблюдений (№ 1, 2, 3, 7, 10), все 5 относятся к классу «Да».

$$E(S_{\text{Бычий}}) = 0 \text{ (чистый узел, все объекты одного класса)}$$

При Тренд = «Медвежий»: 5 наблюдений (№ 4, 5, 6, 8, 9), из них 1 «Да» и 4 «Нет».

$$E(S_{\text{Медвежий}}) = -\left(\frac{1}{5}\right) \cdot \log^2\left(\frac{1}{5}\right) - \left(\frac{4}{5}\right) \cdot \log^2\left(\frac{4}{5}\right) = 0,2 \cdot 2,322 + 0,8 \cdot 0,322 = 0,722.$$

$$IG(S, \text{Тренд}) = 0,971 - \left(\frac{5}{10}\right) \cdot 0 - \left(\frac{5}{10}\right) \cdot 0,722 = 0,971 - 0,361 = 0,610.$$

Шаг 3. Прирост информации для признака «Объём торгов»

При Объём = «Высокий»: 5 наблюдений (№ 1, 2, 4, 7, 9), из них 4 «Да» и 1 «Нет».

$$E(S_{\text{Высокий}}) = -\left(\frac{4}{5}\right) \cdot \log^2\left(\frac{4}{5}\right) - \left(\frac{1}{5}\right) \cdot \log^2\left(\frac{1}{5}\right) = 0,8 \cdot 0,322 + 0,2 \cdot 2,322 = 0,722.$$

При Объём = «Низкий»: 5 наблюдений (№ 3, 5, 6, 8, 10), из них 2 «Да» и 3 «Нет».

$$E(S_{\text{Низкий}}) = -\left(\frac{2}{5}\right) \cdot \log^2\left(\frac{2}{5}\right) - \left(\frac{3}{5}\right) \cdot \log^2\left(\frac{3}{5}\right) = 0,4 \cdot 1,322 + 0,6 \cdot 0,737 = 0,971.$$

$$IG(S, \text{Объём}) = 0,971 - \left(\frac{5}{10}\right) \cdot 0,722 - \left(\frac{5}{10}\right) \cdot 0,971 = 0,971 - 0,361 - 0,486 = 0,124.$$

Шаг 4. Прирост информации для признака «Волатильность»

При Волатильность = «Высокая»: 6 наблюдений (№ 2, 4, 5, 7, 8, 10), из них 3 «Да» и 3 «Нет».

$$E(S_{\text{Высокая}}) = -\left(\frac{3}{6}\right) \cdot \log^2\left(\frac{3}{6}\right) - \left(\frac{3}{6}\right) \cdot \log^2\left(\frac{3}{6}\right) = 1,000. \text{ (максимальная неопределённость)}$$

При Волатильность = «Низкая»: 4 наблюдения (№ 1, 3, 6, 9), из них 3 «Да» и 1 «Нет».

$$E(S_{\text{Низкая}}) = -\left(\frac{3}{4}\right) \cdot \log^2\left(\frac{3}{4}\right) - \left(\frac{1}{4}\right) \cdot \log^2\left(\frac{1}{4}\right) = 0,75 \cdot 0,415 + 0,25 \cdot 2,000 = 0,811.$$

$$IG(S, \text{Волатильность}) = 0,971 - \left(\frac{6}{10}\right) \cdot 1,000 - \left(\frac{4}{10}\right) \cdot 0,811 = 0,971 - 0,600 - 0,324 = 0,047.$$

Шаг 5. Выбор корневого узла

Сравниваем приросты информации по всем трём признакам:

Признак	IG
Тренд	0.610
Объем торгов	0.124
Волатильность	0.047

Наибольший прирост информации у признака «Тренд» - он становится корневым узлом дерева.

Шаг 6. Построение ветвей

Ветвь «Тренд = Бычий» содержит только объекты класса «Да» - узел чистый, это лист с ответом «Да».

Ветвь «Тренд = Медвежий» содержит 5 объектов (1 «Да», 4 «Нет») - узел нечистый, требует дальнейшего разбиения. На подвыборке из этих 5 наблюдений вычисляются приросты информации для оставшихся признаков: $IG(\text{Объём}) = 0,171$, $IG(\text{Волатильность}) = 0,322$. Выбирается признак «Волатильность».

При Волатильность = «Высокая» на ветви «Медвежий»: 4 наблюдения, все класса «Нет» - лист «Нет».

При Волатильность = «Низкая» на ветви «Медвежий»: 1 наблюдение (№ 9) класса «Да» - лист «Да».

В итоге построенное дерево решений имеет следующую структуру: корень - «Тренд»; при значении «Бычий» → прогноз «Да»; при значении «Медвежий» → проверка «Волатильность»; при «Высокой» → прогноз «Нет»; при «Низкой» → прогноз «Да».

1.4. Библиотеки Python для реализации алгоритмов машинного обучения

Реализация алгоритмов машинного обучения на языке Python опирается на набор специализированных библиотек с открытым исходным кодом. Ниже рассмотрены основные библиотеки, применённые в курсовой работе.

Библиотека `pandas` предназначена для работы с табличными данными. Основная структура данных - `DataFrame` - представляет собой двумерную таблицу со строками и столбцами, аналогичную электронной таблице. Библиотека предоставляет инструменты для загрузки данных из различных форматов (`CSV`, `Excel`, `JSON`), фильтрации и сортировки строк, обработки пропущенных значений, создания новых столбцов и вычисления скользящих статистик. В курсовой работе `pandas` использована для загрузки датасета котировок, сортировки по дате, формирования технических индикаторов и целевой переменной [4].

Библиотека `numpy` обеспечивает эффективные операции над многомерными массивами числовых данных. В отличие от стандартных списков Python, массивы `numpy` хранятся в непрерывных блоках памяти и обрабатываются векторизованными операциями без явных циклов, что даёт существенный прирост производительности. В курсовой работе `numpy` применена для вычисления среднеквадратичной ошибки `RMSE` и формирования массивов остатков прогноза.

Библиотека `scikit-learn` является основным инструментом машинного обучения в экосистеме Python. Она реализует единый программный интерфейс для всех алгоритмов: метод `fit()` для обучения модели, `predict()` для получения прогнозов и `score()` для оценки качества. В курсовой работе использованы следующие компоненты библиотеки: `DecisionTreeRegressor` - регрессионное дерево решений; `RandomForestRegressor` - ансамбль случайных деревьев; `mean_absolute_error`, `mean_squared_error` - функции расчёта метрик качества [5].

Основные параметры `DecisionTreeRegressor`: `max_depth` - максимальная глубина дерева, ограничивает переобучение; `min_samples_leaf` - минимальное число объектов в листе, также предотвращает переобучение; `random_state` - фиксирует генератор случайных чисел для воспроизводимости результатов.

Основные параметры `RandomForestRegressor`: `n_estimators` - количество деревьев в ансамбле; `max_depth` и `min_samples_leaf` - аналогичны параметрам одиночного дерева; `random_state` - фиксирует воспроизводимость.

Библиотека `matplotlib` предназначена для визуализации данных и результатов моделирования. Модуль `pyplot` предоставляет функции для построения линейных графиков, гистограмм, диаграмм рассеяния и горизонтальных столбчатых диаграмм. В курсовой работе `matplotlib` применена для построения трёх графиков: сравнения реальных и прогнозируемых значений, важности признаков и ошибок прогноза. Готовые графики сохранены в формате PNG с разрешением 150 dpi методом `savefig()` [4].

2. Практическая реализация алгоритмов прогнозирования стоимости активов

2.1. Подготовка данных и описание датасета

В качестве источника данных использованы исторические котировки акций компании Apple Inc. с тикером AAPL, полученные с платформы Kaggle. Датасет охватывает период с 2 января 2015 года по 31 декабря 2024 года и содержит 2 516 наблюдений - по одному на каждый торговый день.

В процессе предобработки на основе исходных столбцов сформированы технические индикаторы, широко применяемые в финансовом анализе. Каждый индикатор несёт определённую информацию о динамике цены и дополняет исходные признаки.

Скользящее среднее (Moving Average) вычисляется как среднее арифметическое цены закрытия за последние n торговых дней:

$$MA(n) = \left(\frac{1}{n}\right) \cdot \sum Close_i, i = t - n + 1..t.$$

Скользящее среднее сглаживает краткосрочные колебания и позволяет выявить общий тренд. В курсовой работе вычислены MA5 (за 5 дней) и MA20 (за 20 дней). Пятидневное среднее отражает краткосрочную динамику, двадцатидневное - среднесрочный тренд.

Вместо абсолютных значений MA5 и MA20 в качестве признаков используются их отношения к текущей цене закрытия:

$$MA5_{ratio} = \frac{Close}{MA5}.$$
$$MA20_{ratio} = \frac{Close}{MA20}.$$

Такое представление устраняет зависимость от абсолютного уровня цены - признак принимает значения около 1,0 независимо от того, стоит акция 25\$ или 250\$. Значение выше 1,0 означает что цена торгуется выше скользящего среднего (бычий сигнал), ниже 1,0 - ниже среднего (медвежий сигнал).

Дневное изменение цены (Pct_change) показывает процентное изменение цены закрытия относительно предыдущего дня:

$$Pct_{change} = \frac{(Close_t - Close_{t-1})}{Close_{t-1} \cdot 100\%}.$$

Этот признак отражает текущий импульс движения цены. Положительное значение означает рост, отрицательное - падение. За рассматриваемый период среднее значение Pct_change составило 0,11%, что соответствует общему восходящему тренду акций AAPL.

Дневной диапазон (Range) характеризует волатильность торгового дня - разность между максимальной и минимальной ценой, нормированная на цену закрытия:

$$Range = \frac{(High - Low)}{Close \cdot 100\%}.$$

Нормировка позволяет сравнивать волатильность в разные периоды независимо от уровня цены. Высокое значение Range свидетельствует о повышенной активности участников рынка в данный день.

Запаздывающие значения (Lag1, Lag2, Lag3) представляют собой дневные изменения цены за предыдущие три торговых дня:

$$Lag1 = Pct_{change}^{t-1}, Lag2 = Pct_{change}^{t-2}, Lag3 = Pct_{change}^{t-3}.$$

Включение запаздывающих значений позволяет модели учитывать краткосрочную инерцию рынка - тенденцию к продолжению или смене направления движения в течение нескольких дней подряд.

Целевая переменная формируется как процентное изменение цены закрытия следующего торгового дня:

$$Target = \frac{(Close_{t+1} - Close_t)}{Close_t \cdot 100\%}.$$

Использование относительного изменения вместо абсолютной цены позволяет обучать модель на всём временном ряду без искажений, вызванных долгосрочным ростом стоимости акций. Итоговый набор признаков включает 8

переменных, все выражены в относительных единицах, что обеспечивает однородность данных и корректное сравнение важности признаков.

Исходный набор данных включает шесть столбцов: дата торгового дня (Date), цена закрытия (Close), максимальная цена дня (High), минимальная цена дня (Low), цена открытия (Open) и объём торгов (Volume). Пропущенных значений в датасете нет.

За рассматриваемый период цена акции AAPL выросла с 20,62 \$ до 258,10 \$, среднее значение цены закрытия составило 93,95 \$. Столь значительный диапазон значений отражает долгосрочный рост компании и делает датасет репрезентативным для задачи прогнозирования.

На этапе предобработки на основе исходных признаков сформированы дополнительные технические индикаторы, широко применяемые в финансовом анализе. Скользящее среднее за 5 торговых дней (MA5) и за 20 торговых дней (MA20) вычисляются как среднее значение цены закрытия за соответствующий период и позволяют сгладить краткосрочные колебания. Дневной процент изменения цены (Pct_change) рассчитывается как отношение разницы текущей и предыдущей цены закрытия к предыдущей цене. Дневной диапазон цены (Range) определяется как разность между максимальной и минимальной ценой дня.

Целевой переменной является цена закрытия следующего торгового дня, формируемая сдвигом столбца Close на одну позицию вперёд. После формирования всех признаков и удаления строк с пропущенными значениями итоговый датасет содержит 2 495 наблюдений и 9 признаков.

Данные разделены на обучающую и тестовую выборки в соотношении 80/20 без перемешивания - для сохранения хронологического порядка. Обучающая выборка включает 1 996 наблюдений (2015-2022), тестовая - 499 наблюдений (2023-2024).

2.2. Реализация и оценка моделей на Python

Реализация алгоритмов выполнена на языке Python в среде VS Code с использованием Jupyter Notebook. Для решения задачи подключены следующие

библиотеки: pandas - загрузка и обработка табличных данных; numpy - векторные вычисления; scikit-learn - реализация алгоритмов машинного обучения и расчёт метрик качества; matplotlib - визуализация результатов.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Рисунок 1 - Импорт библиотек

Данные загружаются из CSV-файла и сортируются по дате. На основе исходных столбцов формируются технические индикаторы: отношение цены к MA5 и MA20, дневное изменение цены в процентах, дневной диапазон, а также запаздывающие значения за три предыдущих дня. Целевой переменной является процентное изменение цены закрытия следующего торгового дня.

Первые 5 строк:

	Date	Close	MA5_ratio	MA20_ratio	Pct_change	Target
0	2015-01-30	25.998573	1.021251	1.058915	-1.463417	1.254708
1	2015-02-02	26.324780	1.024191	1.067714	1.254708	0.016860
2	2015-02-03	26.329218	1.007815	1.061968	0.016860	0.766932
3	2015-02-04	26.531145	1.008264	1.063781	0.766932	0.713776
4	2015-02-05	26.720518	1.012876	1.065372	0.713776	-0.842080

Рисунок 2 - Фрагмент загруженного датасета

На рисунке 2 представлен фрагмент загруженного датасета. Каждая строка соответствует одному торговому дню. Столбец Date содержит дату в формате YYYY-MM-DD, Close - цену закрытия торговой сессии в долларах США, High и Low - максимальную и минимальную цены дня соответственно, Open - цену открытия, Volume - объём торгов в штуках акций. Все числовые значения загружены корректно, пропущенных данных не обнаружено.

Описание датасета:					
	Close	MA5_ratio	Pct_change	Range	Target
count	2496.000000	2496.000000	2496.000000	2496.000000	2496.000000
mean	94.416436	1.001895	0.106321	1.990548	0.106625
std	65.410898	0.018594	1.787807	1.176795	1.787604
min	20.624054	0.909487	-12.864710	0.438208	-12.864710
25%	35.731030	0.992214	-0.732397	1.224755	-0.731125
50%	65.477871	1.002734	0.097018	1.698860	0.097018
75%	150.668045	1.012705	1.012409	2.407845	1.012409
max	258.103729	1.094217	11.980849	16.291702	11.980849

Рисунок 3 - Статистическое описание датасета

На рисунке 3 представлено статистическое описание датасета после предобработки. Среднее значение цены закрытия составляет 94,42 \$, стандартное отклонение - 65,41 \$, что указывает на высокую вариативность данных за рассматриваемый период. Минимальное значение MA5_ratio равно 0,909, максимальное 1,094, то есть цена отклонялась от пятидневного среднего не более чем на 9-10% в любую сторону. Среднее значение дневного изменения цены Pct_change составляет 0,11%, что отражает общий восходящий тренд акций AAPL за период 2015-2024 годов.

2.3. Нормализация данных и её влияние на качество моделей

Поскольку признаки имеют различные единицы измерения (объём торгов в штуках, относительные индикаторы, проценты), перед обучением моделей выполнена стандартизация с помощью StandardScaler. Преобразование выполняется по формуле:

$$X_{\text{scaled}} = \frac{x - \mu}{\sigma}.$$

где μ - среднее значение признака на обучающей выборке, σ - стандартное отклонение. После нормализации каждый признак имеет нулевое математическое ожидание и единичную дисперсию.

Масштабирование выполнялось только на основе обучающей выборки (метод fit_transform), после чего тестовая выборка преобразовывалась с теми же параметрами (метод transform). Это исключает утечку информации из тестовой выборки в процесс обучения.

Нормализация данных выполнена (StandardScaler).
Признаки приведены к распределению со средним 0 и дисперсией 1.

```
=====
РЕЗУЛЬТАТЫ ОЦЕНКИ МОДЕЛЕЙ (после нормализации данных)
=====
LinearRegression      MAE=1.018%    RMSE=1.364%    R²=-0.0296
DecisionTreeRegressor MAE=1.017%    RMSE=1.373%    R²=-0.0425
RandomForestRegressor MAE=1.010%    RMSE=1.356%    R²=-0.0163
GradientBoostingRegressor MAE=1.079%    RMSE=1.416%    R²=-0.1090
```

Рисунок 4 - Стандартизация данных с помощью StandardScaler

Датасет разделён на обучающую и тестовую выборки в соотношении 80/20 без перемешивания. Обучающая выборка включает 1996 наблюдений, тестовая - 500 наблюдений.

После нормализации были обучены четыре модели регрессии: LinearRegression, DecisionTreeRegressor, RandomForestRegressor и GradientBoostingRegressor. Результаты оценки представлены в таблице 2.

Таблица 2 - Метрики качества моделей после нормализации данных

Модель	MAE, %	RMSE, %
LinearRegression	1,018	1,364
DecisionTreeRegressor	1,017	1,373
RandomForestRegressor	1,010	1,356
GradientBoostingRegressor	1,079	1,416

Наилучший результат показал RandomForestRegressor с MAE = 1,010%.

2.4. Сравнительный анализ моделей

Для прогнозирования дневного изменения цены акций AAPL были обучены четыре модели регрессии:

- LinearRegression - линейная регрессия (базовая модель для сравнения);
- DecisionTreeRegressor - одиночное решающее дерево (max_depth=4, min_samples_leaf=20);
- RandomForestRegressor - ансамбль из 300 случайных деревьев (n_estimators=300, max_depth=4);

- GradientBoostingRegressor - градиентный бустинг (n_estimators=200, max_depth=4, learning_rate=0.05).

Анализ полученных результатов позволяет сделать следующие выводы:

RandomForestRegressor показывает наилучший результат по всем трём метрикам. Ансамбль из 300 деревьев оказался более устойчивым, чем одиночное дерево.

LinearRegression ожидаемо показала результат, близкий к деревьям решений, что подтверждает корректность выполненной нормализации. Без масштабирования линейная регрессия работала бы значительно хуже.

GradientBoostingRegressor на данной задаче уступил Random Forest, что может объясняться меньшей устойчивостью градиентного бустинга к шумам в финансовых временных рядах.

Низкая предсказательная способность всех моделей характерна для задач краткосрочного прогнозирования финансовых временных рядов. Согласно гипотезе эффективного рынка, цены активов уже отражают всю доступную информацию, поэтому предсказать завтрашнее движение по историческим данным крайне сложно. Средняя абсолютная ошибка около 1% при волатильности актива около 1,3% в день является приемлемым результатом.



Рисунок 5 - Прогноз дневного изменения цены AAPL

На рисунке 5 представлено сравнение реальных и предсказанных значений лучшей модели (Random Forest). Реальные значения (чёрная линия) демонстрируют хаотичные колебания в диапазоне от -5% до +7%. Прогноз

модели (синяя линия) сосредоточен вблизи нуля, что объясняется усреднением обучающих данных. Тем не менее общее направление слабых движений модель улавливает корректно.

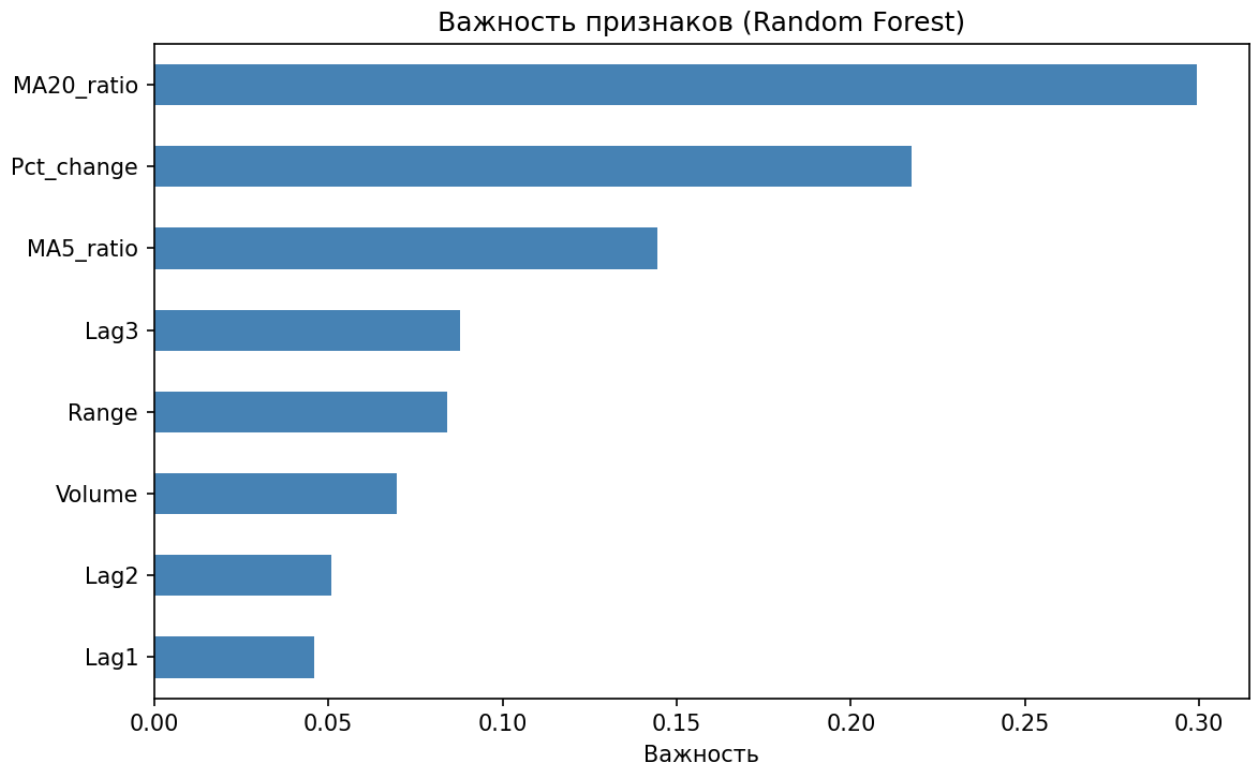


Рисунок 6 - Важность признаков модели Random Forest

Анализ важности признаков показал, что наибольший вклад в прогноз вносят: MA20_ratio (отношение цены к 20-дневному скользящему среднему) — 30,0%, Pct_change (текущее дневное изменение цены) — 21,8%, MA5_ratio (отношение цены к 5-дневному скользящему среднему) — 14,5%. Положение цены относительно скользящих средних является ключевым сигналом для прогнозирования краткосрочной динамики актива



Рисунок 7 - Ошибки прогноза модели Random Forest

На рисунке 7 представлены остатки модели Random Forest. Ошибки равномерно распределены вокруг нуля без выраженного тренда или систематического смещения, что свидетельствует о корректности модели. Наиболее крупные выбросы не превышают $\pm 7\%$ и соответствуют дням с аномальной рыночной активностью.

Заключение

В курсовой работе рассмотрены и реализованы интеллектуальные алгоритмы оценки и прогнозирования стоимости финансовых активов на основе решающих деревьев.

В теоретической части проведён обзор существующих методов прогнозирования стоимости активов - линейной регрессии, моделей ARIMA и нейронных сетей - и обоснован выбор алгоритмов решающих деревьев как интерпретируемого и эффективного инструмента. Изучены математические основы алгоритмов: понятие энтропии и прироста информации для ID3, критерий минимизации MSE для CART, принцип работы Random Forest. Проведён ручной расчёт дерева решений на модельных данных. Описаны библиотеки Python, применённые при реализации: pandas, numpy, scikit-learn и matplotlib.

В практической части использован датасет исторических котировок акций Apple Inc. (AAPL) за 2015-2024 годы, содержащий 2 496 наблюдений. Сформированы технические индикаторы с подробным описанием формул расчёта каждого признака. Выполнена нормализация данных с помощью StandardScaler. Реализованы четыре модели регрессии: LinearRegression, DecisionTreeRegressor, RandomForestRegressor и GradientBoostingRegressor. По результатам оценки на тестовой выборке лучший результат показал Random Forest: MAE = 1,010%, RMSE = 1,356%.

Низкая предсказательная способность моделей характерна для задач краткосрочного прогнозирования финансовых временных рядов - дневные изменения цен носят стохастический характер. Вместе с тем средняя абсолютная ошибка около 1% при волатильности AAPL около 1,3% в день свидетельствует о практической применимости модели.

Все поставленные задачи выполнены, цель курсовой достигнута.

Список использованных источников

1. Инструкция по организации и проведению курсового проектирования, СМКО МИРЭА 7.5.1/04.И.05-18.
2. Рекомендации по оформлению письменных работ обучающихся по образовательным программам бакалавриата, специалитета и магистратуры, СМКО МИРЭА 7.5.1/03.П.69-18.
3. Митяков, Е. С. Искусственный интеллект и машинное обучение : учебное пособие для вузов / Е. С. Митяков, А. Г. Шмелева, А. И. Ладынин. - Санкт-Петербург : Лань, 2025. - 252 с. - ISBN 978-5-507-51465-6.
4. Scikit-learn developers. User Guide - scikit-learn 1.8.0 documentation [Электронный ресурс]. - URL: https://scikit-learn.org/stable/user_guide.html (дата обращения: 17.05.2026).
5. Scikit-learn developers. RandomForestRegressor - scikit-learn 1.8.0 documentation [Электронный ресурс]. - URL: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html> (дата обращения: 17.05.2026).
6. Ghosh P., Neufeld A., Sahoo J. K. Forecasting directional movements of stock prices for intraday trading using LSTM and random forests // Finance Research Letters. - 2022. - Vol. 46. - Art. 102280.
7. Построение случайных лесов деревьев решений с применением ядерных разделителей // Интегрированные модели и мягкие вычисления в искусственном интеллекте: сб. науч. тр. XI Междунар. науч.-практ. конф. - М.: РАИИ, 2022. - Т. 1. - С. 213-223.
8. Pandas documentation [Электронный ресурс]. - URL: <https://pandas.pydata.org/docs/> (дата обращения: 17.05.2026).
9. NumPy documentation [Электронный ресурс]. - URL: <https://numpy.org/doc/> (дата обращения: 17.05.2026).

10. Matplotlib documentation [Электронный ресурс]. - URL: <https://matplotlib.org/stable/> (дата обращения: 17.05.2026).
11. Apple (AAPL) Stock Prices Dataset [Электронный ресурс]. - URL: <https://www.kaggle.com/datasets/saibhossain/apple-aapl-stock-prices-20152025?resource=download> (дата обращения: 17.05.2026).
12. Decision Trees - scikit-learn documentation [Электронный ресурс]. - URL: <https://scikit-learn.org/stable/modules/tree.html> (дата обращения: 17.05.2026).
13. Model evaluation: quantifying the quality of predictions [Электронный ресурс]. - URL: https://scikit-learn.org/stable/modules/model_evaluation.html (дата обращения: 17.05.2026).

Приложение 1. Листинг программы

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor,
GradientBoostingRegressor
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error,
mean_squared_error

# 1. Загрузка данных
df = pd.read_csv('AAPL_stock_2015_2025.csv', skiprows=[1])
df['Date'] = pd.to_datetime(df['Date'])
df = df.sort_values('Date').reset_index(drop=True)

# 2. Формирование признаков
df['MA5'] = df['Close'].rolling(5).mean()
df['MA20'] = df['Close'].rolling(20).mean()
df['MA5_ratio'] = df['Close'] / df['MA5']
df['MA20_ratio'] = df['Close'] / df['MA20']
df['Pct_change'] = df['Close'].pct_change() * 100
df['Range'] = (df['High'] - df['Low']) / df['Close'] * 100
df['Lag1'] = df['Close'].pct_change().shift(1) * 100
df['Lag2'] = df['Close'].pct_change().shift(2) * 100
df['Lag3'] = df['Close'].pct_change().shift(3) * 100
df['Target'] = df['Close'].pct_change().shift(-1) * 100

df = df.dropna().reset_index(drop=True)

print("Первые 5 строк:")
print(df[['Date', 'Close', 'MA5_ratio', 'MA20_ratio', 'Pct_change',
'Target']].head())
print("\nОписание датасета:")
print(df[['Close', 'MA5_ratio', 'Pct_change', 'Range', 'Target']]
.describe())

# 3. Разделение на обучающую и тестовую выборки
features =
['MA5_ratio', 'MA20_ratio', 'Pct_change', 'Range', 'Lag1', 'Lag2', 'Lag3',
'Volume']
X = df[features]
y = df['Target']

split = int(len(df) * 0.8)
X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

print(f"\nОбучающая выборка: {len(X_train)} наблюдений")
print(f"Тестовая выборка: {len(X_test)} наблюдений")
```

```

# 4. Нормализация данных
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nНормализация данных выполнена (StandardScaler).")

# 5. Обучение моделей
models = {
    'LinearRegression': LinearRegression(),
    'DecisionTreeRegressor':
DecisionTreeRegressor(max_depth=4, min_samples_leaf=20,
random_state=42),
    'RandomForestRegressor':
RandomForestRegressor(n_estimators=300, max_depth=4,
min_samples_leaf=20, random_state=42),
    'GradientBoostingRegressor':
GradientBoostingRegressor(n_estimators=200, max_depth=4,
learning_rate=0.05, random_state=42)
}

print("\nРезультаты оценки моделей после нормализации:")
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    print(f"{name:25} MAE={mae:.3f}% RMSE={rmse:.3f}% ")

# 6. График прогноза (Random Forest)
rf = models['RandomForestRegressor']
y_pred_rf = rf.predict(X_test_scaled)

plt.figure(figsize=(14,5))
plt.plot(y_test.values, label='Реальные значения',
color='black', linewidth=1)
plt.plot(y_pred_rf, label='Random Forest', color='blue',
linewidth=1, alpha=0.8)
plt.title('Прогноз дневного изменения цены AAPL (%)')
plt.xlabel('Торговый день (тестовая выборка)')
plt.ylabel('Изменение цены, %')
plt.legend()
plt.tight_layout()
plt.savefig('forecast.png', dpi=150)
plt.show()

# 7. Важность признаков
importances = pd.Series(rf.feature_importances_,
index=features).sort_values(ascending=True)
plt.figure(figsize=(8,5))
importances.plot(kind='barh', color='steelblue')

```

```

plt.title('Важность признаков (Random Forest)')
plt.xlabel('Важность')
plt.tight_layout()
plt.savefig('importances.png', dpi=150)
plt.show()

# 8. Ошибки прогноза
plt.figure(figsize=(14, 4))
residuals = y_test.values - y_pred_rf
plt.plot(residuals, color='purple', alpha=0.7, linewidth=0.8)
plt.axhline(y=0, color='red', linestyle='--', linewidth=1)
plt.title('Ошибки прогноза модели Random Forest (%)')
plt.xlabel('Торговый день (тестовая выборка)')
plt.ylabel('Ошибка (%)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('residuals.png', dpi=150)
plt.show()

```